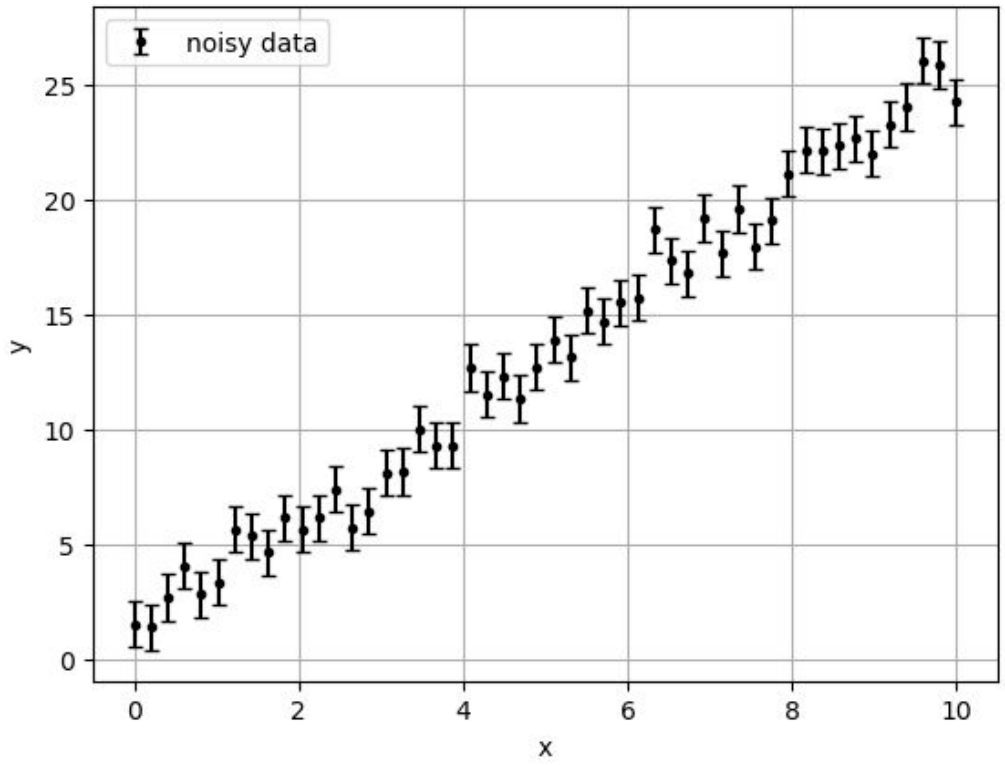
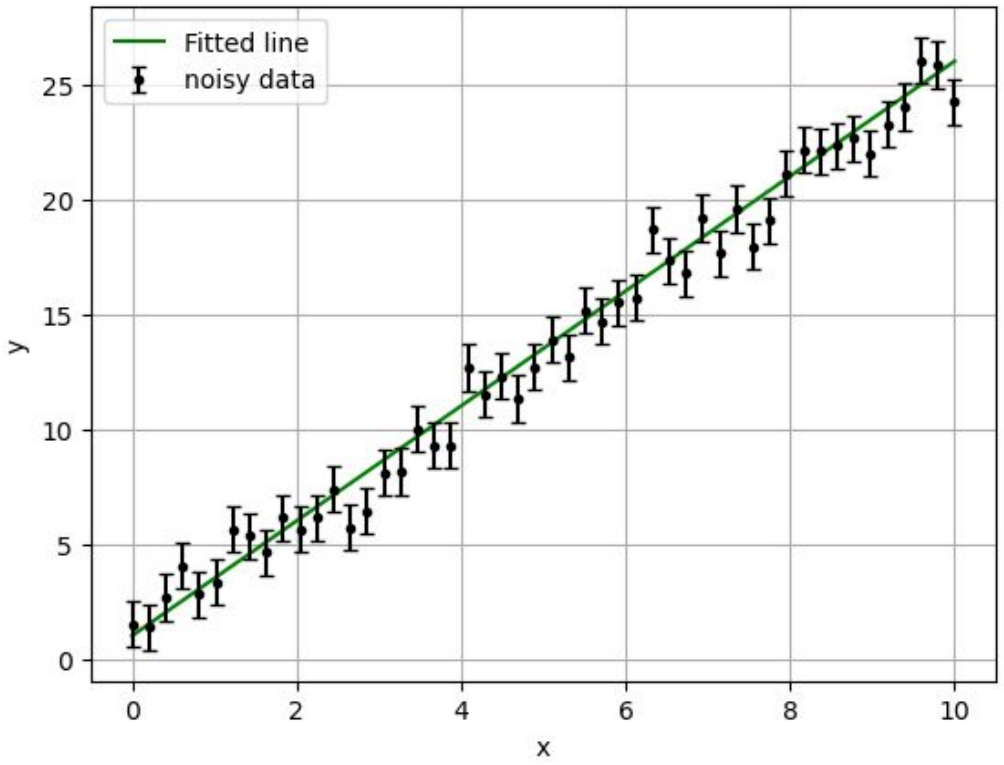
Linear Regression

INTRODUCTION, THEORY, AND APPLICATIONS

In search of the best-fit …





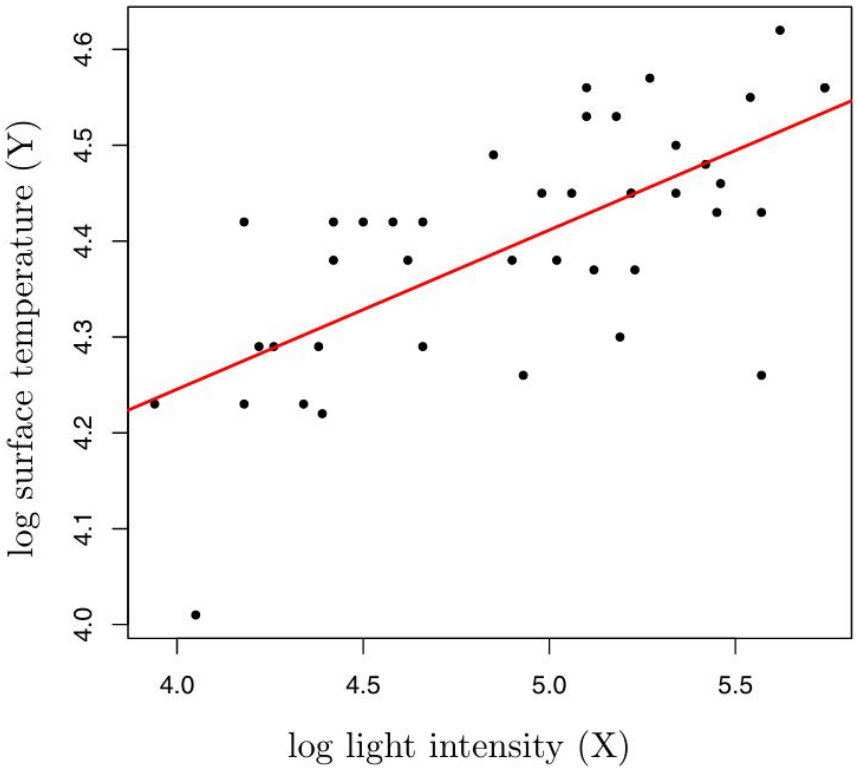
Model y = mx + c

What should be m and c ?

Our goal – To "learn" the relationship between different variables

Linear Regression

- A **supervised machine learning algorithm** used to ***find the best-fitting linear relationship (function)*** between input and output variables in a labeled dataset.
- It assumes that the output (dependent variable) changes at a constant rate with respect to the input (independent variable), forming a straight-line relationship.
- **Aim : To learn this linear relationship so it can be used to make predictions on new, unseen data.**



Data on Nearby stars. The solid line is a least squared fit line.

A fun fact : The term "regression" is due to Sir Francis Galton (1822-1911) who noticed that tall and short men tend to have sons with heights closer to the mean. He called this "regression towards the mean."

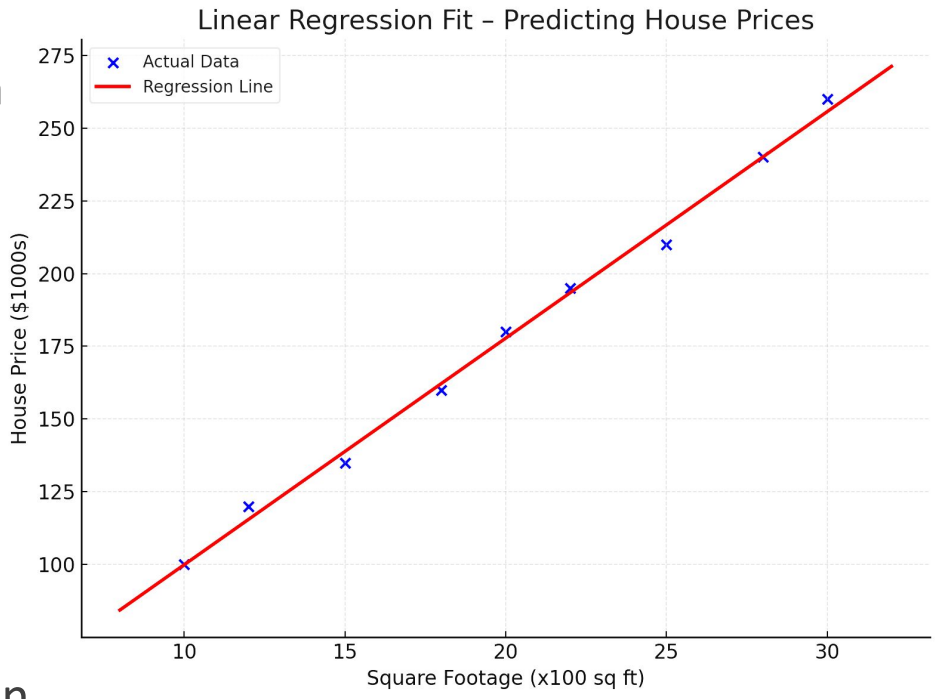
Example

Predicting House Prices

Suppose we want to estimate the price of a house based on its size in square feet. Typically, as the size of a house increases, its price also increases in a somewhat linear fashion.

- **Independent variable (input):** Square footage — this is the factor we measure or observe.
- **Dependent variable (output):** House price — this value depends on the square footage.

The model uses the input (size of the house) to predict the output (house price), assuming a linear relationship between the two.



Importance of Linear Regression

- Simplicity & Interpretability
- Predictive Power

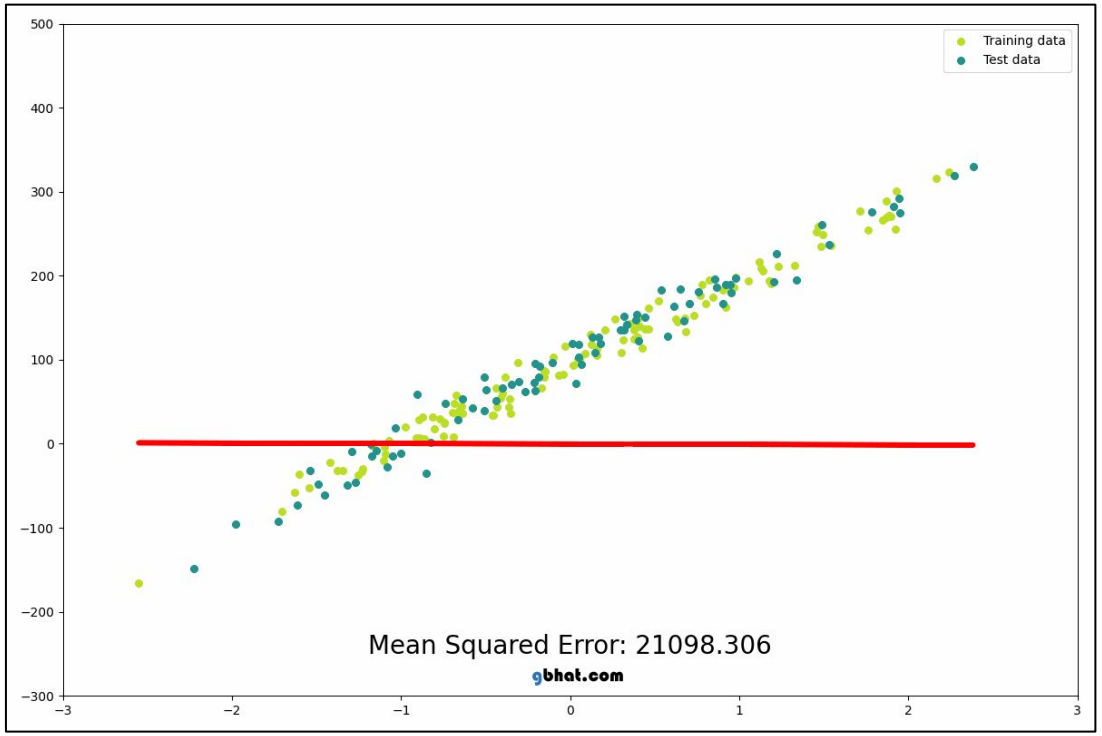
  Enables forecasting based on historical data — useful across industries like finance, healthcare, and marketing.
- Foundation for Advanced Models

  Serves as the building block for more complex techniques (e.g., logistic regression, neural networks).
- Computational Efficiency
- Wide Applicability

  One of the most commonly used algorithms in both statistics and machine learning for regression problems.
- Insight into Relationships

Model Representations (Single Variable)

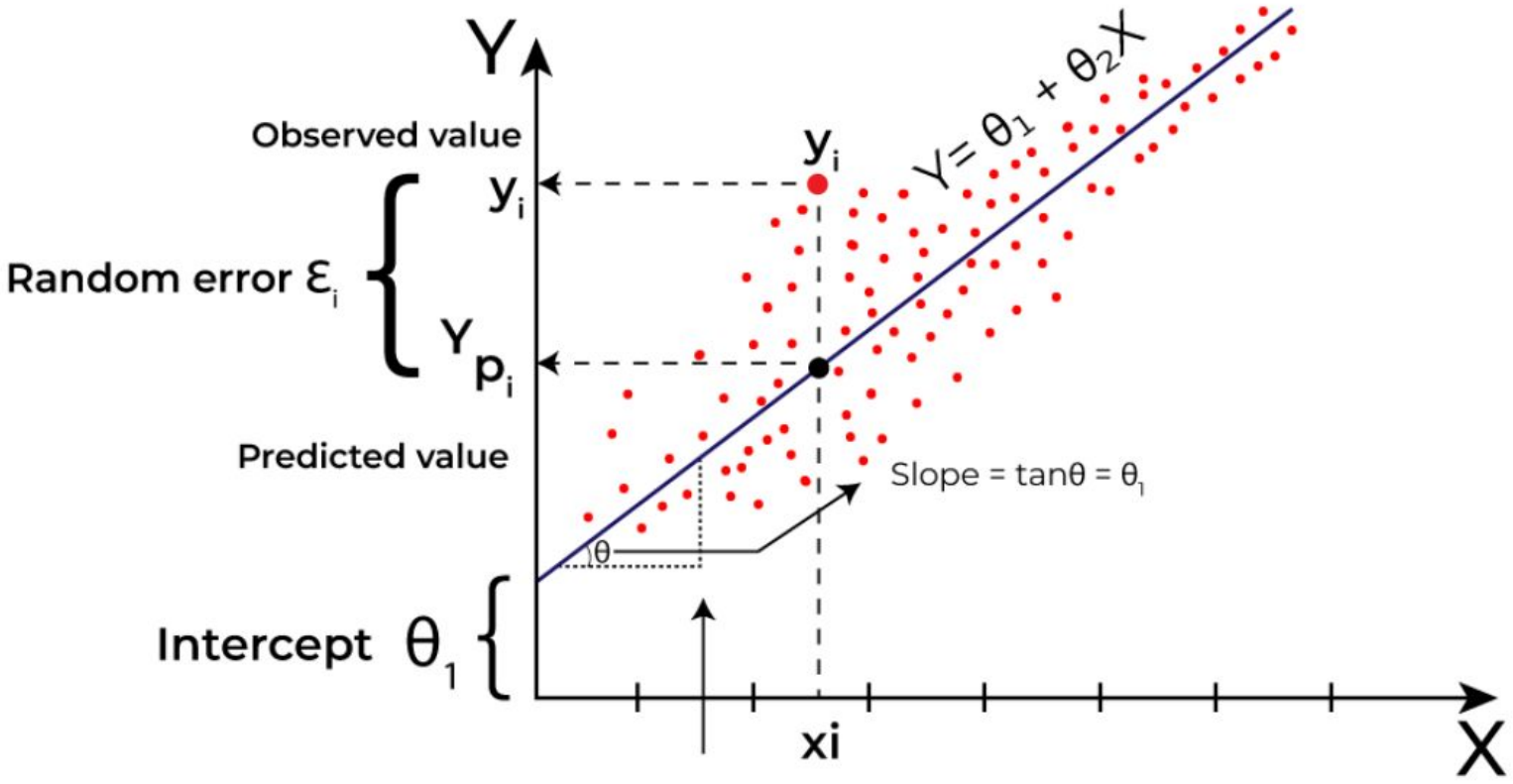
- The model for single variable case is just a simple line.
- Hypothesis/Model : $h(x) = \theta_0 + \theta_1 x$
  - θ_0 => intercept
  - θ_1 => slope
- The job of the linear regression process is to **figure out the best possible value of θ_0 and θ_1 which defines the best fit line**.
- What does the algorithm do ?
  - Start with a random guess of θ_0 and θ_1.
  - Keep adjusting θ_0 and θ_1 till you get the best-fit line.



Source - https://gbhat.com/machine_learning/linear_regression.html

What defines the “best fit” ?

Cost/Loss Function



The difference between predicted value and observed value is quantified by the Loss function.

The "best-fit" line/model is defined by line/model for which the Loss function is minimum.

This implies that, **"learning" is nothing but minimizing the Loss function on training data**.

Cost/Loss Function

In linear regression, there are four main types of loss, which are outlined in the following table.

Loss type	Definition	Equation
L_1 loss	The sum of the absolute values of the difference between the predicted values and the actual values.	$\sum \lvert actual\ value - predicted\ value \rvert$
Mean absolute error (MAE)	The average of L_1 losses across a set of *N* examples.	$\frac{1}{N} \sum \lvert actual\ value - predicted\ value \rvert$
L_2 loss	The sum of the squared difference between the predicted values and the actual values.	$\sum (actual\ value - predicted\ value)^2$
Mean squared error (MSE)	The average of L_2 losses across a set of *N* examples.	$\frac{1}{N} \sum (actual\ value - predicted\ value)^2$

Question : How do we choose the right loss function for a given task ?

But,
How to minimise the Loss function ?

The Gradient Descent

- The Gradient Descent : Climbing down the Valley of “Gradient of Loss Function”.

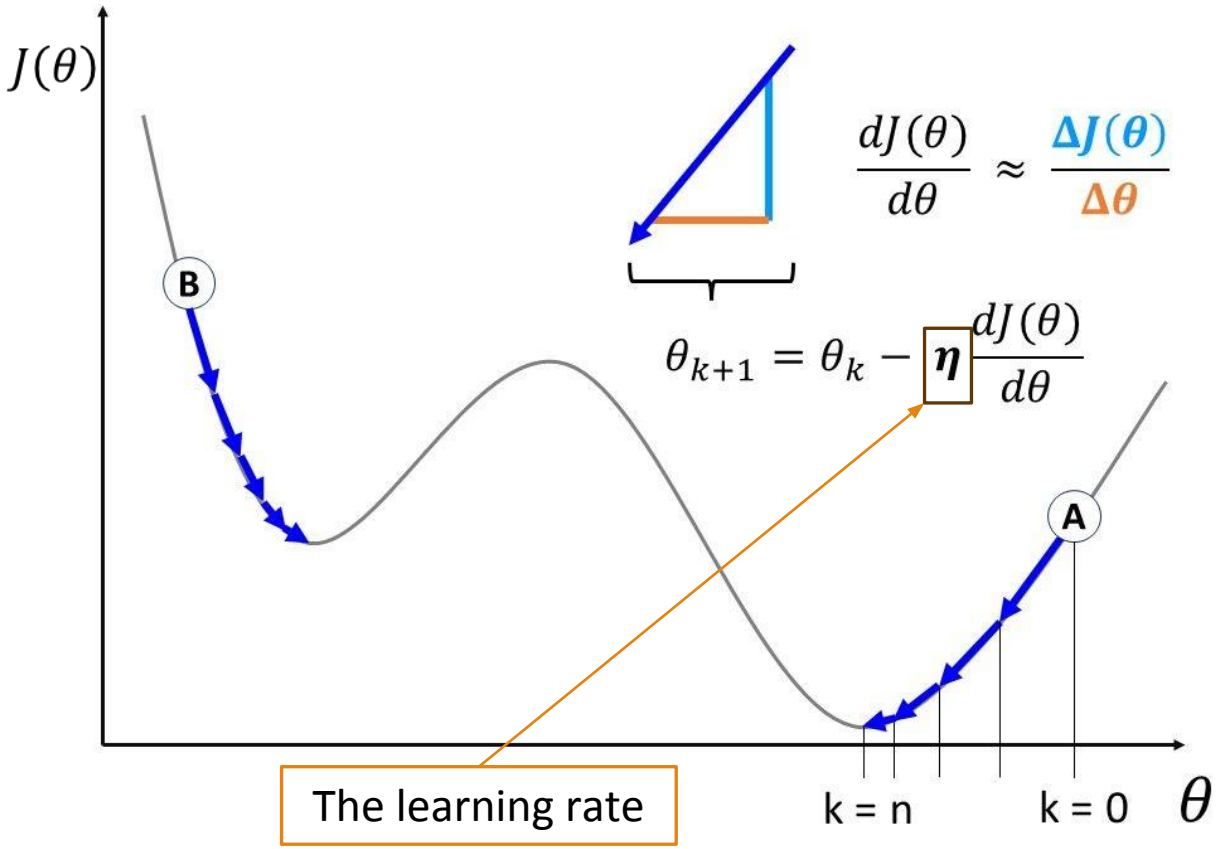
$$D_c = \frac{\partial(Cost\ Function)}{\partial c} = \frac{\partial}{\partial c}\left(\frac{1}{n}\sum_{i=0}^{n}\left(y_i - y_{i\,pred}\right)^2\right)$$

$$= \frac{1}{n}\frac{\partial}{\partial c}\left(\sum_{i=0}^{n}\left(y_i - (mx_i + c)\right)^2\right)$$

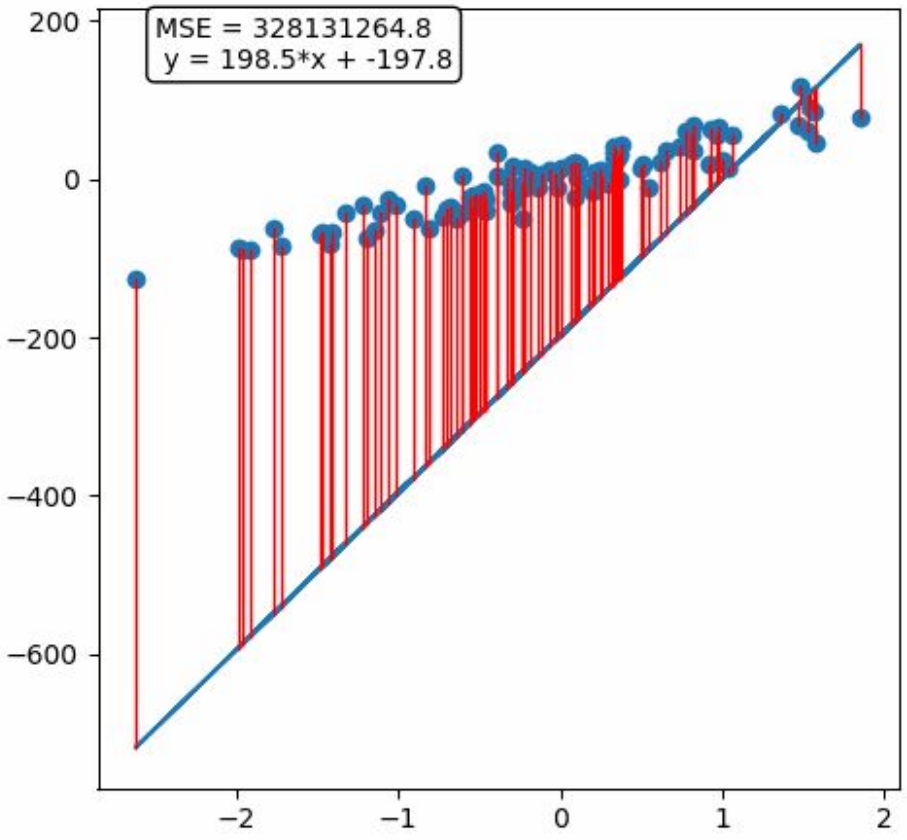
$$= \frac{1}{n}\frac{\partial}{\partial c}\left(\sum_{i=0}^{n}\left(y_i^2 + m^2x_i^2 + c^2 + 2mx_ic - 2y_imx_i - 2y_ic\right)\right)$$

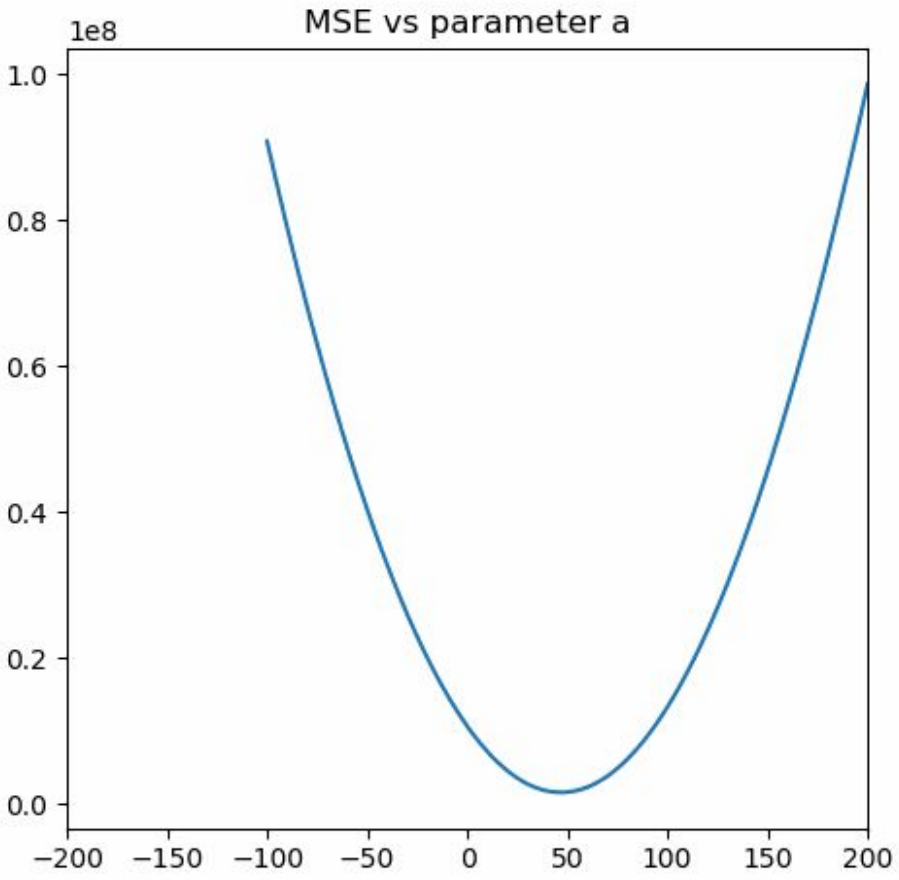
$$= \frac{-2}{n}\sum_{i=0}^{n}\left(y_i - (mx_i + c)\right)$$

$$\frac{-2}{n}\sum_{i=0}^{n}\left(y_i - y_{i\,pred}\right)$$

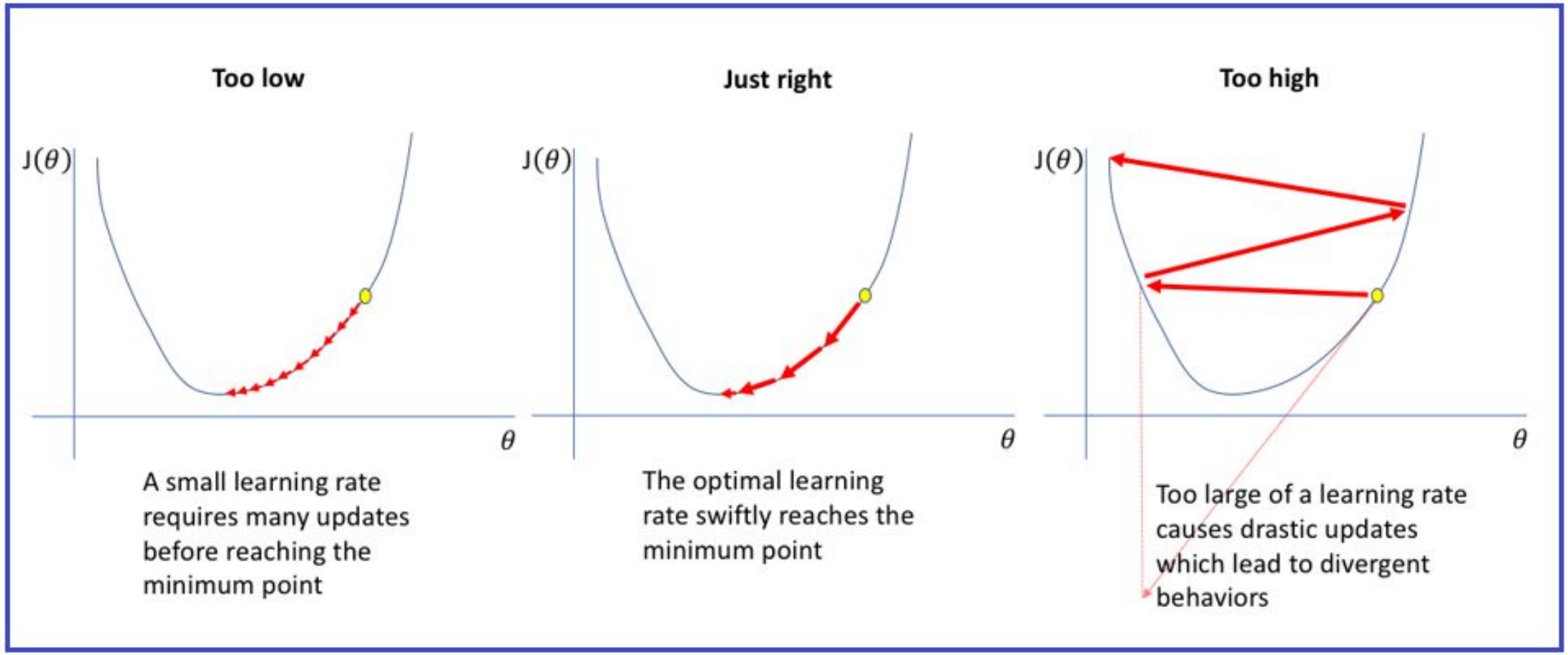


Gradient Descent in action





Impact of learning rate



What if there are more parameters ?

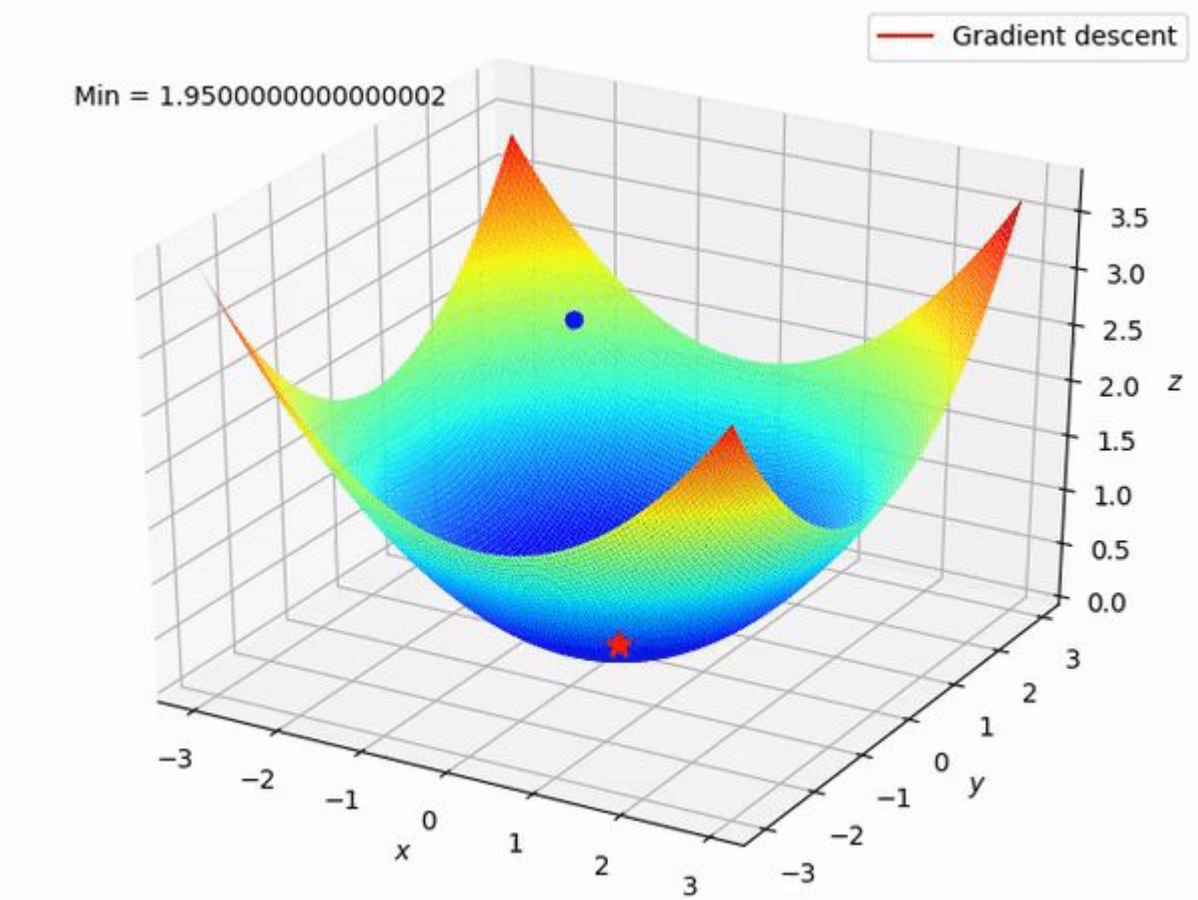
Need a complex model with more than two parameter

The Multivariable Model in LR

- Hypothesis/Model :
  $h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \ \ldots . + \theta_n x_n$
  - θ_0 => bias
  - θ_i => coefficients

- The Loss function becomes multi-dimensional.

- Computationally expensive.



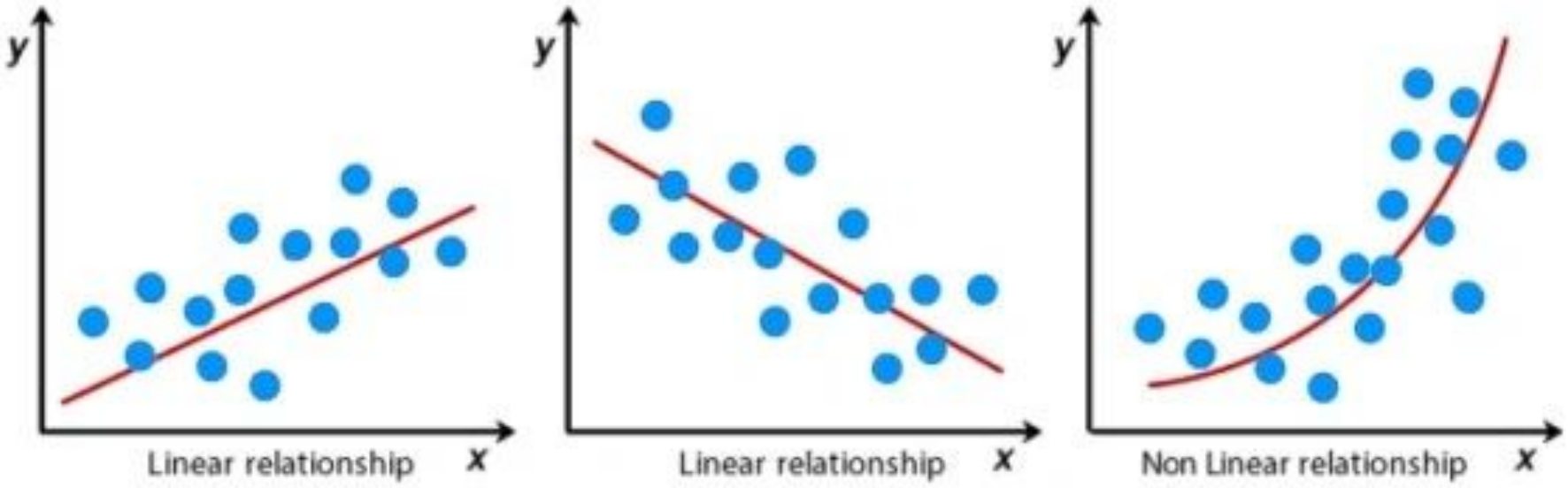
Need to be aware of the assumptions …

- Linearity
- Independence
- Homoscedasticity
- Normality

<u>If any one of the assumption fails for any case then Linear regression will not give appropriate prediction.</u>

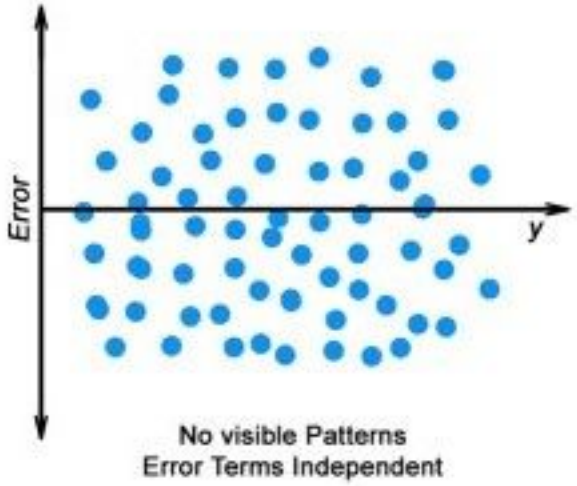
Linearity of Residuals

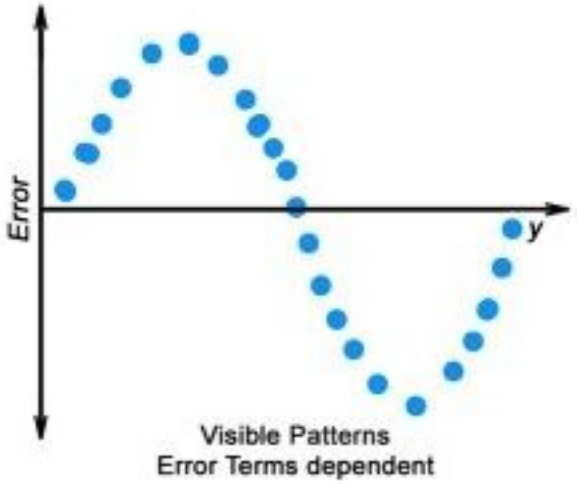
- Linearity
  - There must be a linear relationship between the independent and dependent variables.
  - The residuals (errors) should show no patterns when plotted — a random scatter indicates a good linear fit.



Independence of Residuals

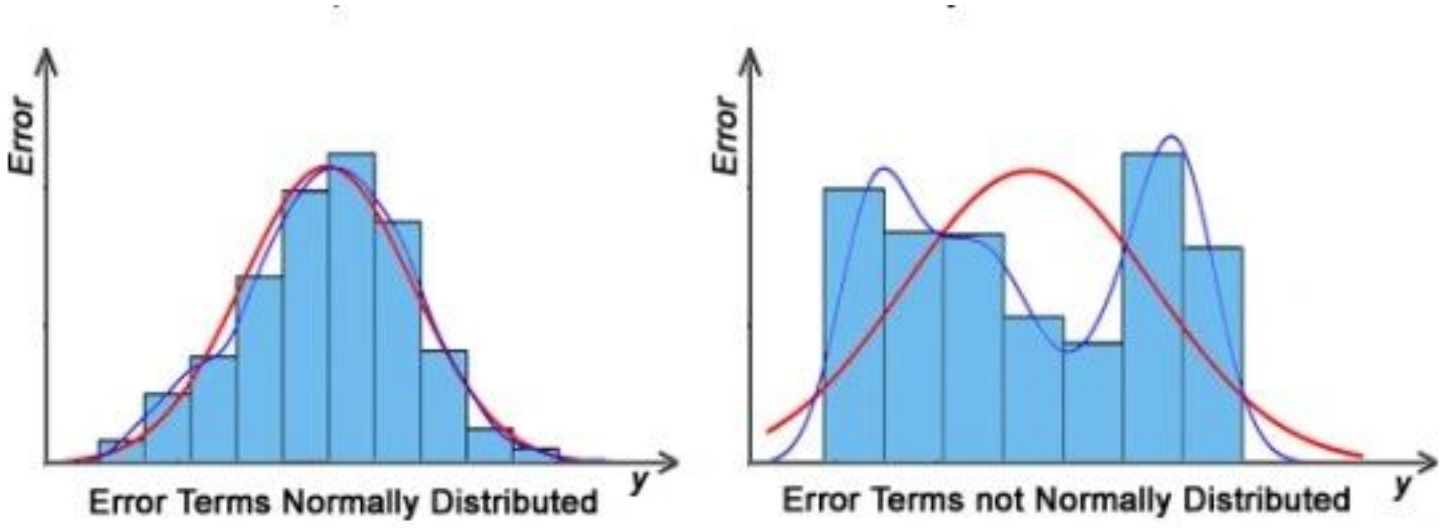
- Independence
  - Error terms should be independent of each other.
  - In time-series or sequential data, where one value may influence the next, this assumption can be violated (known as autocorrelation).
  - No visible structure or pattern in residuals indicates independence.





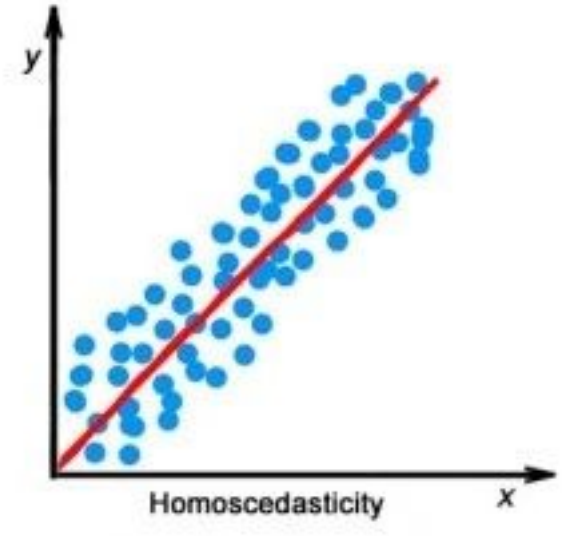
Normality of Residuals

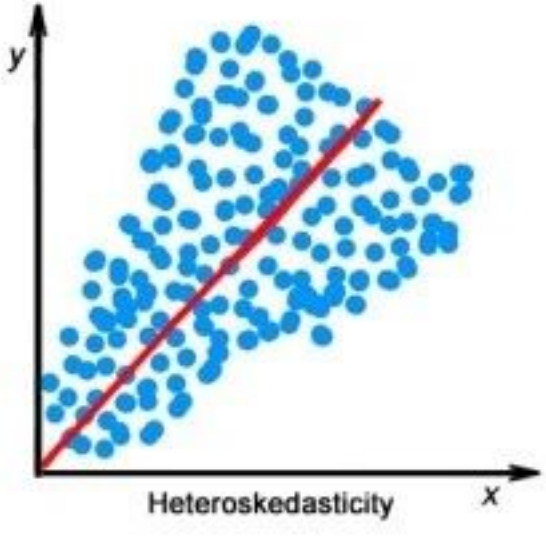
- Normality
  - The residuals should follow a normal distribution centered around zero.
  - This ensures the model's predictions are unbiased and reliable.
  - Significant deviations might suggest the presence of outliers or model misspecification.



Constant Variance of Residuals (Homoscedasticity)

- Homoscedasticity
  - The spread of residuals should remain constant across all levels of the independent variables.
  - If the variance increases or decreases with input values (called heteroscedasticity), it can affect the accuracy and reliability of predictions.
  - Often caused by outliers or extreme values.





Linear Regression - Overview

- A supervised machine learning algorithm used for predicting continuous outcomes
- Learns from labeled data to find the best-fitting straight line between input and output variables
- Assumes a linear (straight-line) relationship: output changes at a constant rate with input
- The learned linear function can be used to make predictions on new data

How do I know how well the model has learnt ?

Evaluation Metric for Linear Regression

- There are many ways to evaluate the performance of a Linear Regression model. We will discuss three of them.

- Coefficient of determination or - value
- Root Mean Squared Value (RMSE)
- Residual Standard Error (RSE)

Coefficient of determination

- R-squared (R^2) measures how much variation in the data is explained by the model.
  - Value ranges between 0 and 1
  - Higher R-squared => better model fit

$$R^2 = 1 - \frac{RSS}{TSS}$$

$$RSS = \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i)^2$$

The sum of squares of the residual for each data point in the plot/data

$$TSS = \sum_{i=1}^{n} (y_i - \bar{y}_i)^2$$

The sum of errors of the data points from the mean of the response variable

RMSE and RSE

- Root Mean Squared Error (RMSE)
  - Measures **absolute fit** — how close predictions are to observed values

$$RMSE = \sqrt{\frac{RSS}{n}} = \sqrt{\frac{\sum_{i=1}^{n}\left(y_i^{actual} - y_i^{predicted}\right)^2}{n}}$$

- Residual Standard Error (RSE)
  - RMSE with adjustment for degrees of freedom ($df = n - 2$)

$$RMSE = \sqrt{\frac{RSS}{n-2}} = \sqrt{\frac{\sum_{i=1}^{n}\left(y_i^{actual} - y_i^{predicted}\right)^2}{n-2}}$$

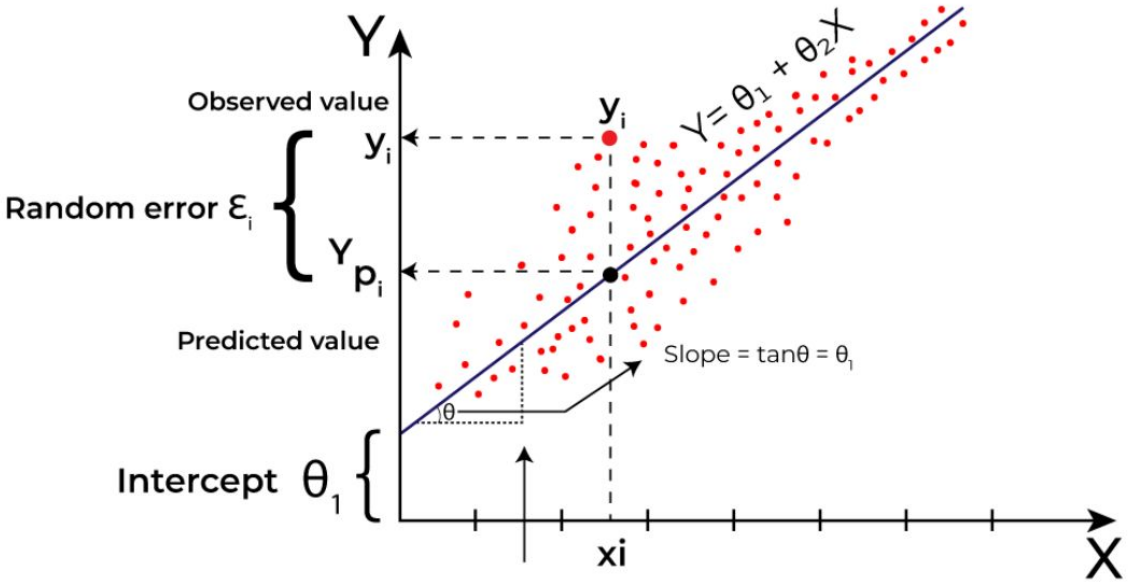
- RMSE is unit-dependent → changes with variable units
- R^2- value preferred → normalized, easier comparison across models

Key point to be noted

“ RMSE gives a sense of prediction accuracy in the same units as the dependent variable. RSE corrects RMSE for degrees of freedom, making it unbiased. However, since RMSE changes with unit scaling, R-squared often provides a more reliable model comparison. ”

Properties of a Regression Line

- Passes through the mean of X and mean of Y
- Regression residuals have a mean of zero
- Slope represents change in Y per unit change in X
- Minimizes the sum of squared residuals (Least Squares Property)
- Regression line of Y on X is different from X on Y (unless correlation = ±1)
- Slope sign matches the sign of the correlation coefficient



Multiple Linear Regression (MLR)

- It is statistical method to model relationship between one dependent variable (Y) and multiple independent variables $(X_1, X_2, X_3, \dots, X_n)$

$$\hat{Y} = \theta_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3 + \ \dots \theta_n X_n + \boxed{\epsilon}$$

Error

- Used when factors collectively influence Y

- MLR allows us to account for multiple factors simultaneously, improving accuracy and understanding of relationships. Each coefficient shows the effect of one variable while keeping others constant (a.k.a. partial derivatives).

Interpretation of coefficients

- **Model :** $\hat{Y} = \theta_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3 + \ \ldots \theta_n X_n + \epsilon$
- **Intercept ($\boldsymbol{\theta_0}$) :** Expected Y when all X's are zero.
- **Slope ($\boldsymbol{\theta_i}$) :** Change in Y per unit change in X_i, holding other variables constant

- **For example :**

- If θ_3 = 5, Y increases by 5 units for each 1-unit increase in X_3, holding other variables fixed.

Assumptions of MLR

- **Linearity:** Relationship between predictors & Y is linear
- **Independence:** Observations are independent
- **Homoscedasticity:** Constant variance of residuals
- **Normality:** Residuals are normally distributed
- **No Multicollinearity:** Predictors are not highly correlated

Similar to that of Linear Regression with two parameters (θ_0, θ_1)

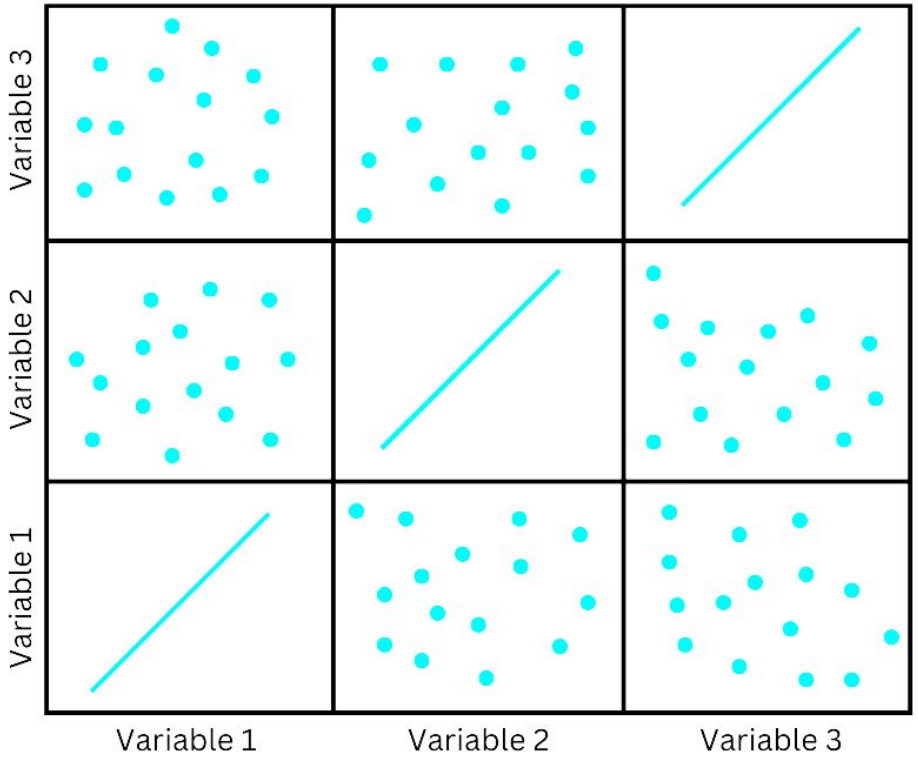
Violations of these assumptions can bias results, inflate errors, or lead to misleading inferences.

Things to be careful about when using MLR

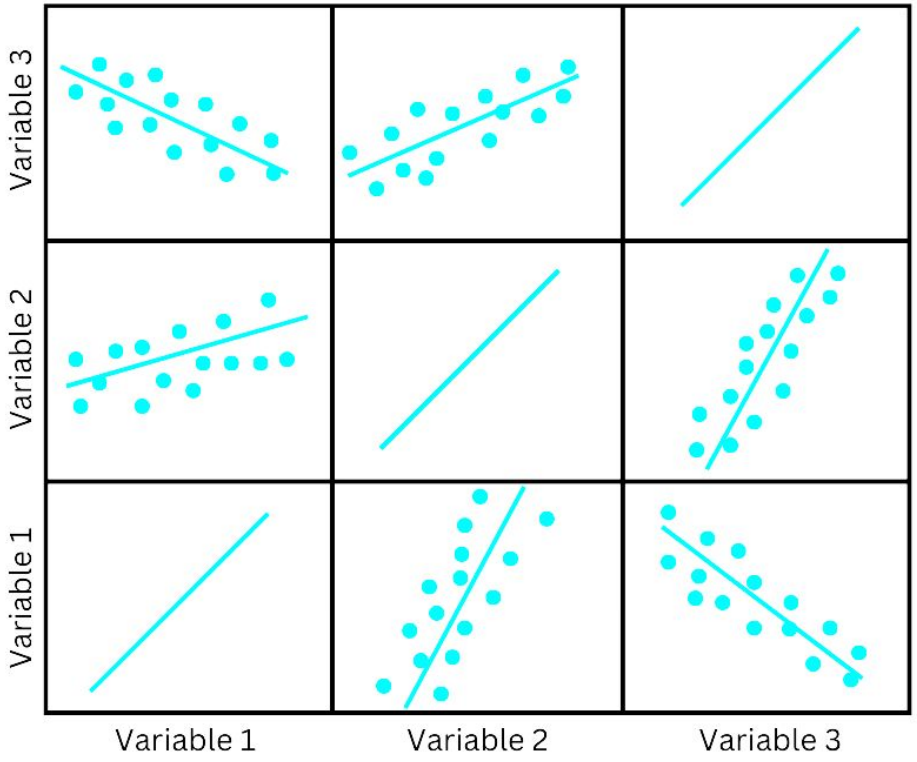
- Apart from the assumptions, we should be careful about the following :
- **Overfitting:**
  - Adding too many predictors → overly complex model
  - Model “memorizes” training data → high training accuracy, low test accuracy
- **Multicollinearity:**
  - Independent variables are highly correlated with each other
  - Leads to unstable coefficient estimates
- **Feature Selection:**
  - Identify most relevant predictors from many available variables
  - Remove redundant or irrelevant features for better model performance

Overfitting harms generalization, multicollinearity makes results unreliable, and feature selection becomes critical when handling large predictor sets.

Multicollinearity



No Multicollinearity



Multicollinearity

Multicollinearity and its detection

- Multicollinearity is a statistical phenomenon that occurs when **two or more independent variables in a regression model are highly correlated, indicating a strong linear relationship among the predictor variables.**

- **Impact of Multicollinearity :**
  - Hard to determine individual predictor's contribution
  - May cause incorrect conclusions about variable effects
  - Predictions remain precise, but coefficient estimates become unstable
  - Removing correlated variables randomly → coefficients swing or change sign

- **Detection Methods**

  1. Pairwise Correlation:
     - Check correlations between independent variable pairs
     - Useful for identifying direct strong relationships

Multicollinearity

2. Variance Inflation Factor (VIF):

- Measures how much variance of a coefficient is inflated due to correlation with other predictors

Formula:

$$VIF_i = \frac{1}{1 - R_i^2}$$

R_i^2 is from regressing X_i^2 on all other predictors

Guidelines:

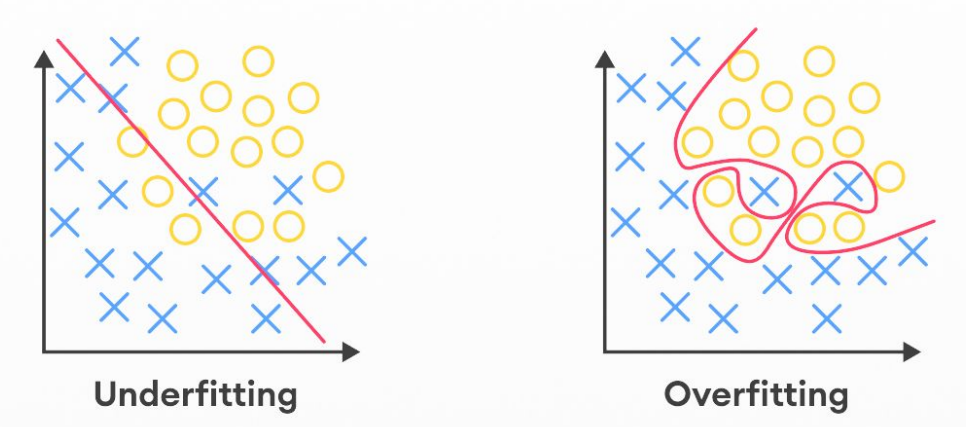
- VIF > 10 → High multicollinearity, consider removal
- VIF ≈ 5 → Inspect carefully
- VIF < 5 → Generally acceptable

Multicollinearity doesn't reduce model accuracy but makes coefficient interpretation unreliable.
VIF is preferred over simple pairwise correlations because it detects combined effects of multiple predictors.
The key is to address it thoughtfully — not by random variable removal.

Overfitting & Underfitting in Linear Regression

- Overfitting:
  - Model performs well on training data but poorly on test data
  - Too complex → memorizes patterns & noise

- Underfitting:
  - Model performs poorly on both training & test data
  - Too simple → fails to capture underlying patterns



Bias and Variance

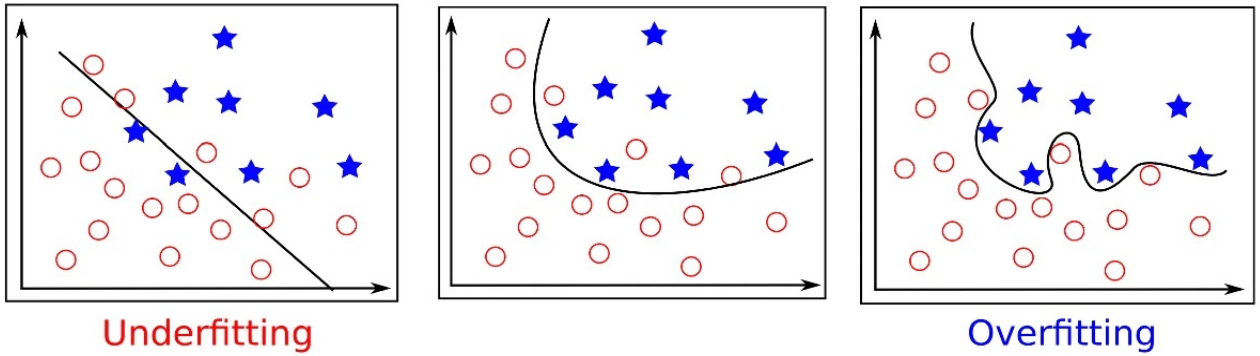
- Bias:
  - Error from overly simplistic model assumptions
  - Bias measures systematic errors
  - High bias → underfitting, low flexibility
  - Linear models often have high bias but are easy to train & interpret

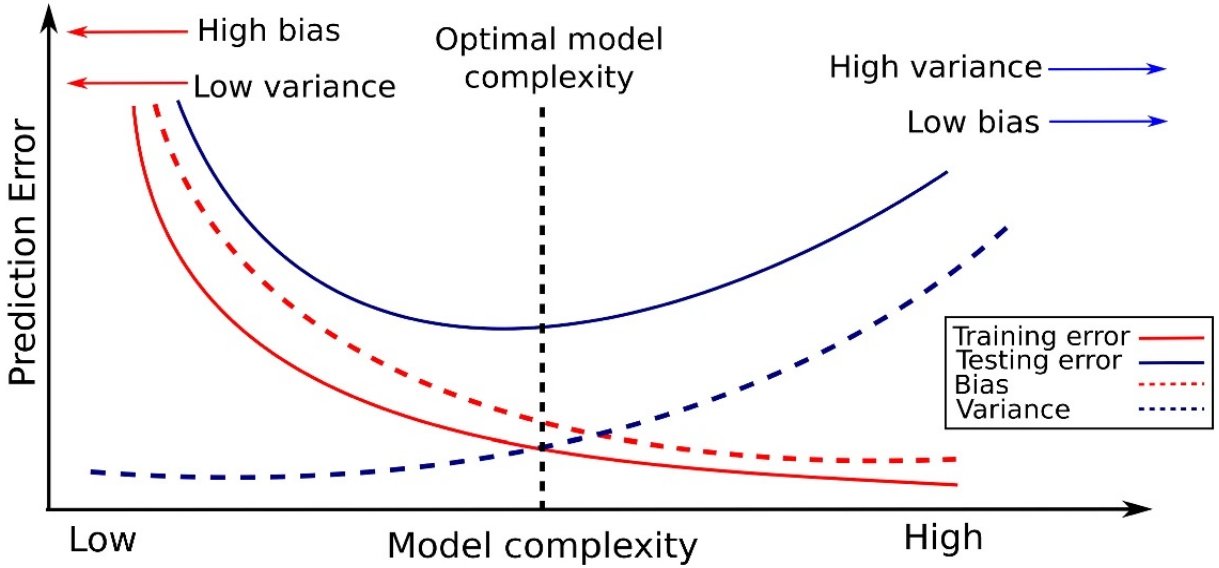
- Variance:
  - Sensitivity of model to training data changes
  - Variance measures instability
  - High variance → overfitting, unstable predictions
  - Low variance → model is stable & generalizable

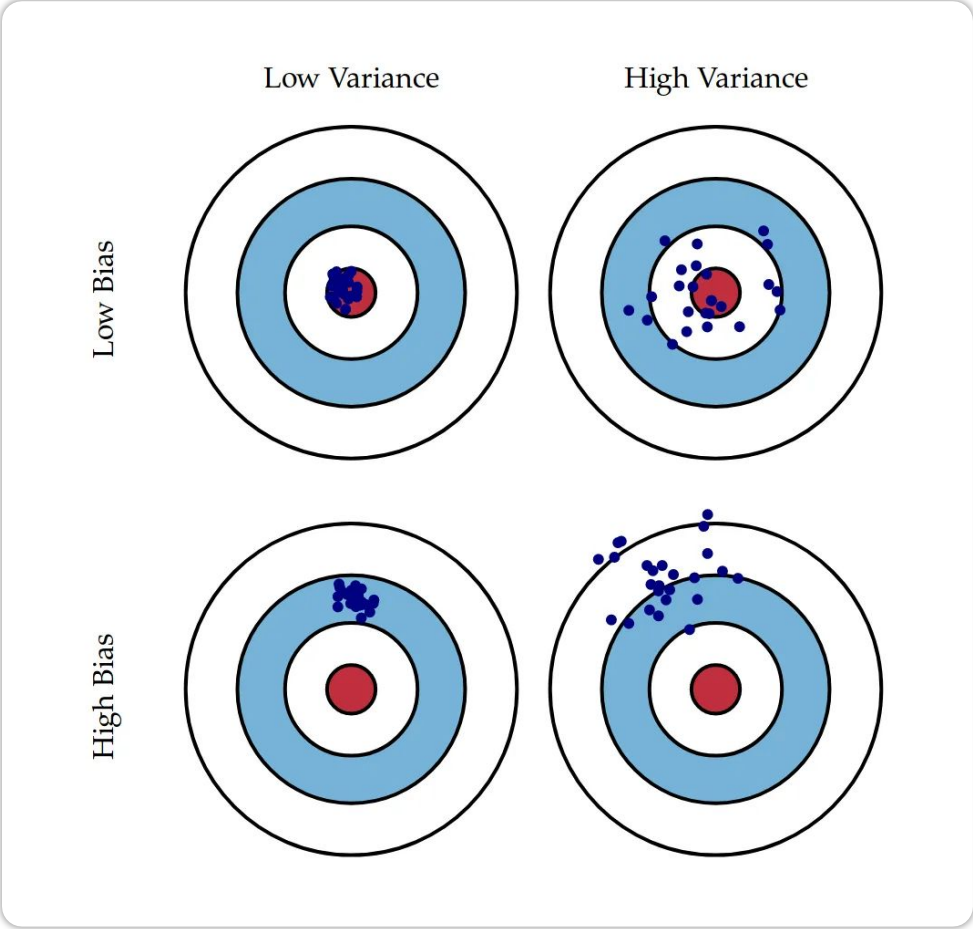
Overfitting happens when variance dominates

Underfitting happens when bias dominates.

The Bias-Variance Trade-off

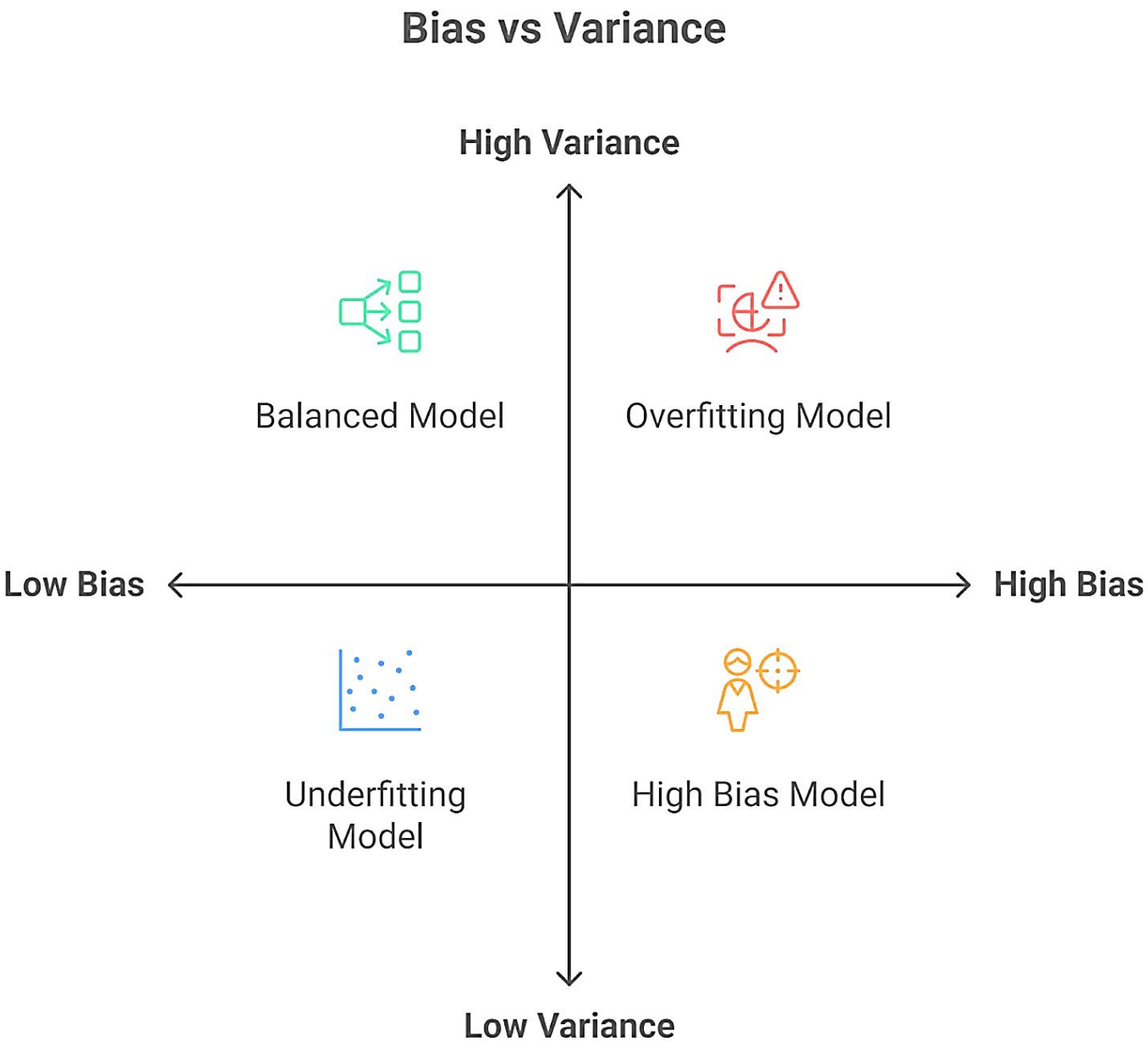






Summary

“ The bias–variance tradeoff is about finding the sweet spot where the model is complex enough to learn patterns but simple enough to generalize.”



So how to handle this bias-variance trade-off ?

Regularization

- Adding a "penalty" term in our cost/loss function. For example, assuming that we are using MSE as our loss (i.e. L2 loss).

- **Purpose:** Prevents overfitting by penalizing large coefficients
  - **L1 (Lasso):** Adds penalty = $\lambda \sum_{i=1}^{n} |\theta_i|$ → feature selection
  - **L2 (Ridge):** Adds penalty = $\lambda \sum_{i=1}^{n} \theta_i^2$ → coefficient shrinkage
  - **Elastic Net:** Combination of L1 & L2 penalties

- **Effect:** Improves generalization by reducing model complexity

New loss = $\frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_0 - \sum_{i=1}^{n}\theta_i x_i)^2 + \lambda \sum_{i=1}^{n}|\theta_i|$

Regularization/Penalty term

New loss = $\frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_0 - \sum_{i=1}^{n}\theta_i x_i)^2 + \lambda \sum_{i=1}^{n}\theta_i^2$

"Regularization introduces a penalty term to the cost function, discouraging overly complex models that fit noise rather than patterns."

Can we use linear regression for classification ?

Issues with Linear Regression in Classification

- Predicts continuous values → not ideal for categorical outcomes
- No probability interpretation → can output values outside [0, 1]
- Sensitive to outliers → distorts classification boundary
- Assumes linear relationship → may not hold in classification problems

For binary outcomes, logistic regression is more suitable because it models probabilities directly and outputs values between 0 and 1.

Logistic Regression

INTRODUCTION, THEORY, AND APPLICATIONS

Recap…

- Issues with Linear Regression for classification tasks

Unbounded Predictions
Linear Regression outputs values across the real line. For classification tasks, this can lead to probabilities <0 or >1, which are invalid in probabilistic interpretation.

Poor Class Separation
Regression line may not separate classes well when data is not linearly separable, leading to incorrect decision boundaries.

Sensitivity to Outliers
Extreme data points can heavily skew the regression line, resulting in unreliable classification predictions.

Illustrative Example
Consider predicting whether a student passes (1) or fails (0) based on study hours. Linear Regression may predict values like -0.3 or 1.2, which do not make sense as probabilities.

Logistic Regression

- Logistic Regression is a supervised machine learning algorithm primarily used for classification problems.
  - It's a statistical method that models the log-odds of an event as a linear combination of one or more independent variables.
  - Its core purpose is to predict the probability that an observation belongs to a specific class or category.
- Key Difference from Linear Regression
  - Unlike linear regression, which predicts continuous values (e.g., house price), logistic regression predicts a categorical outcome.
  - Its output is a **probability between 0 and 1**, rather than an unbounded continuous value.

What does probability has to do with the classification ?

Interpretation of probabilities

- The returned probability can be used "as is" (e.g., 93.2% chance of spam) or converted to a binary category (e.g., "Spam" or "Not Spam") using a threshold (often 0.5).

- Applications (Overview)
  - Widely used in healthcare, finance, marketing, social sciences, and engineering.

Question : Are all classification tasks in ML ultimately are regression tasks based on probabilities ?

“ Classification is regression onto probabilities, followed by a decision rule. ”

*But in practice, classification and regression are considered separate because the learning objectives, metrics, and interpretation differ.

Idea Behind Logistic Regression

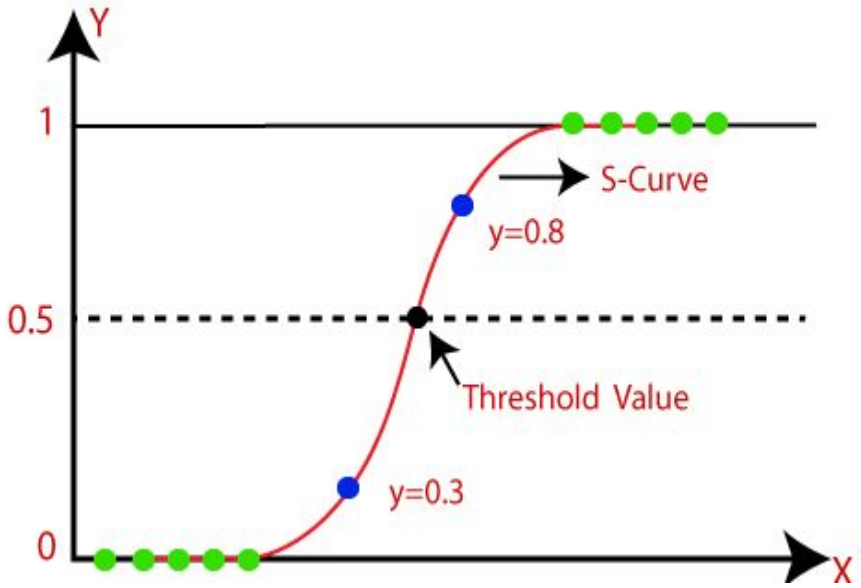
- **Goal :** To predict the probability of a class (e.g., spam = 1).
- Computes a linear combination of inputs:

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

- Pass these linear combination (z) through Sigmoid functior to get output (probability) between 0 and 1.

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Provide a decision rule : Ex => If p > 0.5, predict Class 1; else, Class 0.
- Model is trained by maximizing likelihood (choosing best weights).



Understanding Odds in Classification

- Definition:

$$odds = \frac{p}{1-p}$$

where p = probability of the event.

- Expresses how much more likely an event is to occur than not.

If:

- Odds = 1 → Event is equally likely to happen or not.
- Odds > 1 → Event is more likely to happen.
- Odds < 1 → Event is less likely to happen.

- Logistic regression models the log of odds (logit) as a linear function.

The Logit Function

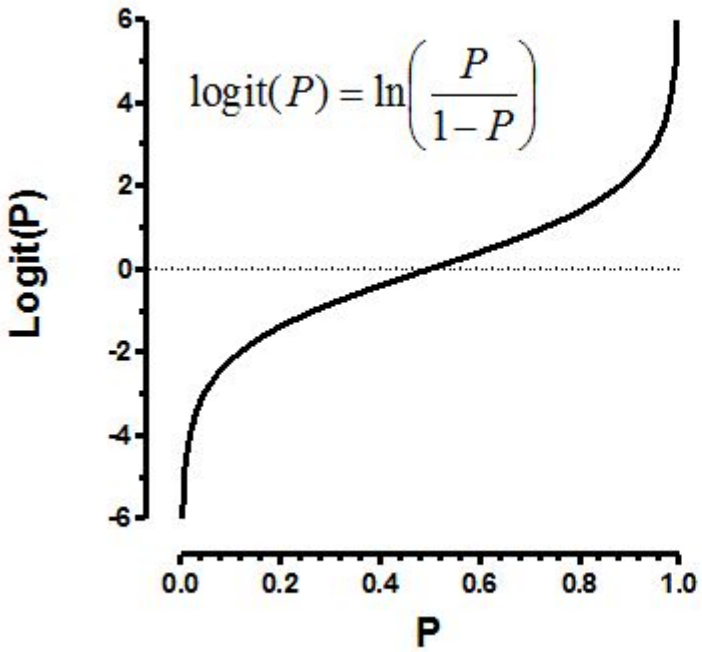
- Logit: The Link Between Odds and Linear Models
- Definition:

$$Logit(p) = \log\left(\frac{p}{1-p}\right)$$

- Transforms probability (0,1) → real numbers (-∞, +∞).
- Makes it possible to model probabilities using linear equations:

$$Logit(p) = \log\left(\frac{p}{1-p}\right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

- Logistic regression is linear in the log-odds, not in raw probability.
- Helps interpret coefficients in terms of how features affect odds.



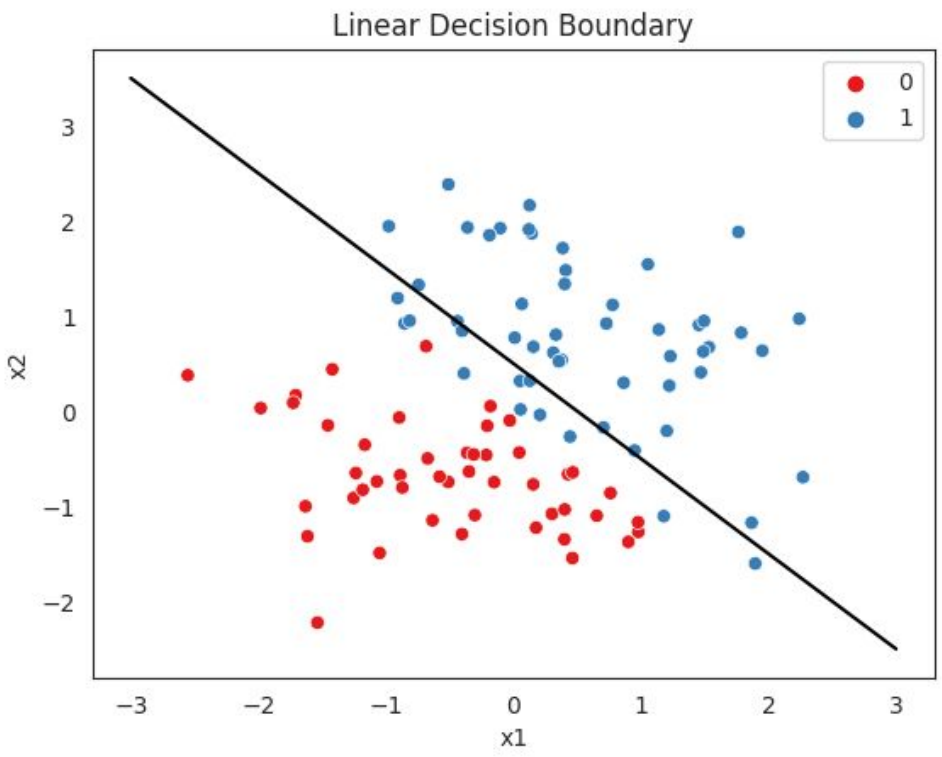
H.W. : Show how Sigmoid and logit functions are related in the context of Logistic regression ?

Decision boundary

- Definition: A line (or surface) that separates classes in feature space.
- In logistic regression:
  - Set threshold at p=0.5.
  - Solve for z = 0 :

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n = 0$$

- Boundary is linear in features (unless features are transformed).
- Points on one side → predict Class 1; other side → Class 0.
- Can extend to higher dimensions (planes, hyperplanes).



The key question : How to optimize the weights for best possible classification ?

Or

How “learning” is done in Logistic Regression ?

Maximum Likelihood: The Core Idea

- **Goal:** Find model parameters (θ_i) that make observed data most probable.
- In logistic regression:
  - For each training point (x_i, y_i), predicted probability:

$$p_i = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n)$$

  - Likelihood of dataset:

$$L(\theta_0, \ldots \theta_n) = \prod_{i=1}^{m} p_i^{y_i}(1 - p_i)^{1-y_i}$$

y_i are class labels (0, 1)

- We want to maximize L(w) to find best-fit weights.
- Easier to work with log-likelihood (sum instead of product).

Log-Likelihood in Logistic Regression

- Log – Likelihood :

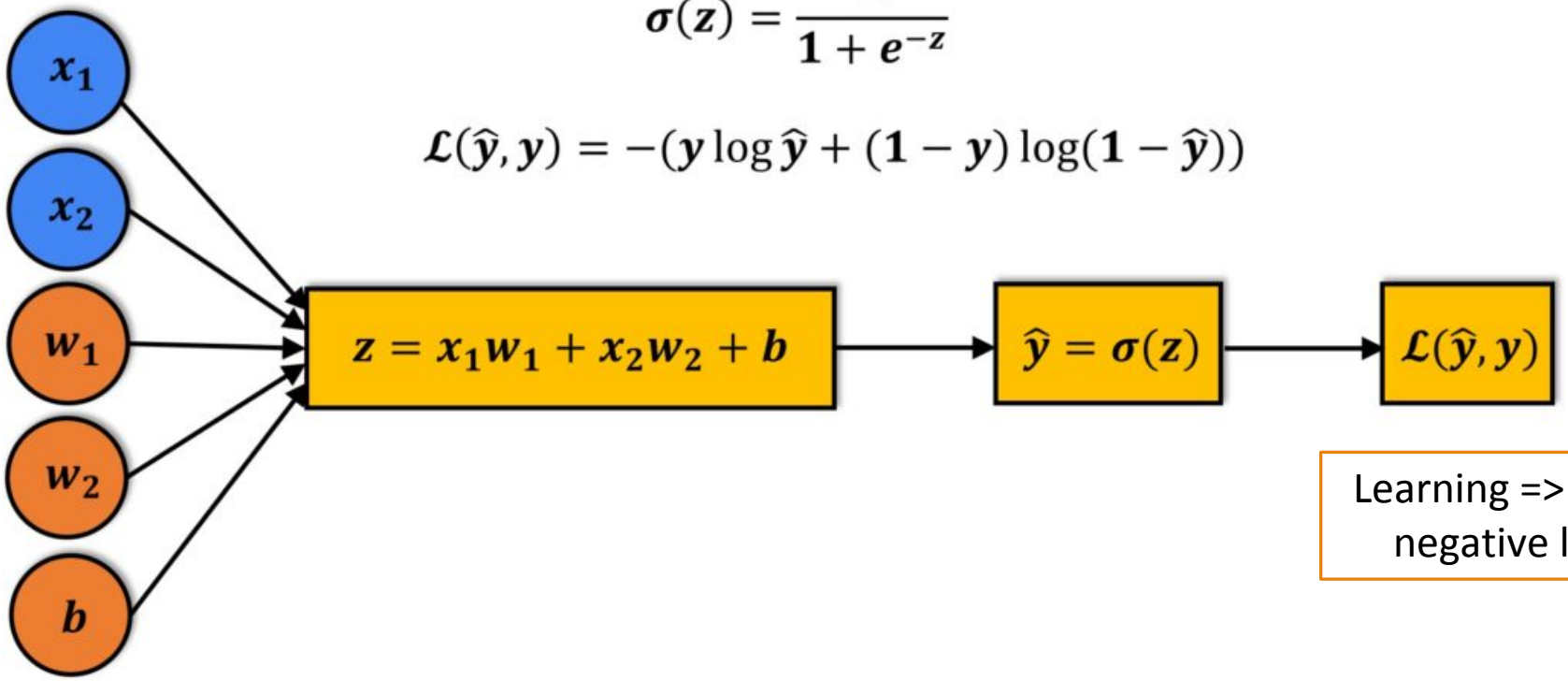
$$l(\theta_0, \dots, \theta_n) = \sum_{i=1}^{m} [\, y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \,]$$

- Turning product into a sum simplifies optimization.
- **Maximize ℓ(w) using gradient ascent** or **minimize negative log-likelihood** (cross-entropy loss).
- MLE provides interpretable coefficients tied to odds.

Overview

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$



Learning => Minimising the negative log-likelihood

Comparison with Linear Regression

Feature	Linear Regression	Logistic Regression
Output	Continuous value ($\mathbb{R}$)	Probability (0 to 1)
Goal	Predict numeric value	Classify into categories
Model	$y = w^T x + b$	$p = \sigma(w^T x + b)$
Decision Boundary	No natural boundary	Probability threshold (e.g., 0.5)
Loss Function	Mean Squared Error (MSE)	Cross-Entropy / Log Loss
Interpretation	Coefficients are slopes	Coefficients affect odds of class
Use Case	Regression tasks	Classification tasks

Pros & Cons of Logistic Regression

✅ Pros	⚠️ Cons
Simple, easy to implement	Assumes linear decision boundary
Outputs **probabilities** (interpretable)	Struggles with **non-linear** relationships
Coefficients are interpretable (odds ratios)	Performance drops with high-dimensional data
Efficient training (convex optimization)	Sensitive to outliers and multicollinearity
Works well on small/medium datasets	Requires **feature scaling** for stability
Extensible to multi-class (One-vs-Rest, Softmax)	Limited modeling capacity vs. complex ML models

NOTE : Logistic Regression is a strong baseline for classification — interpretable, simple, and efficient, but not suited for complex, highly non-linear problems.

Looking back at loss/cost function for logistic regression

Why we suddenly switched to “negative log likelihood” instead of MSE ?

Cost Function : Cross-Entropy

- We minimize negative log-likelihood (**Cross-Entropy Loss or Log Loss**) to train logistic regression :

$$l(\theta_0, \dots, \theta_n) = \sum_{i=1}^{m} [\, y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \,]$$

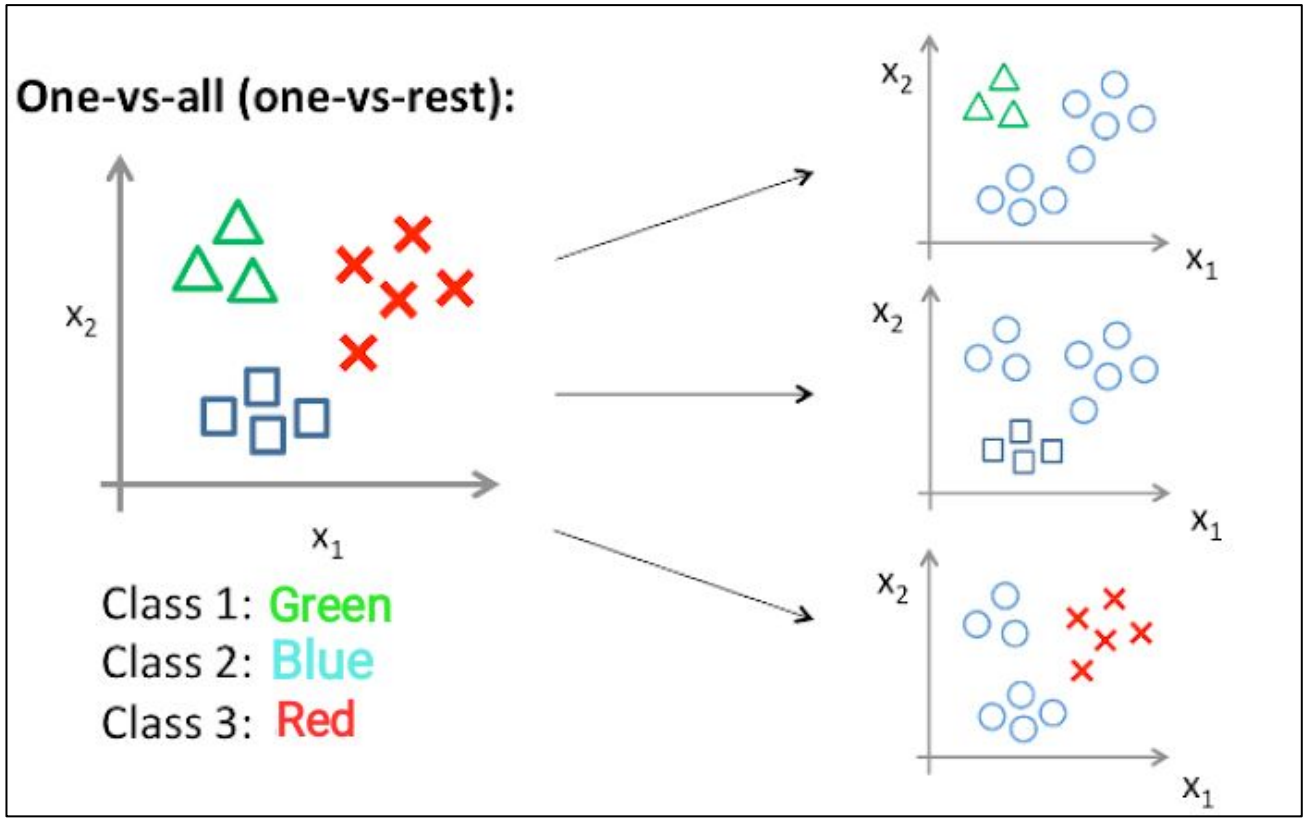
- Why not MSE?
  - MSE leads to non-convex optimization for classification.
  - Cross-entropy is convex, easier to optimize.
  - Loss penalizes confident wrong predictions heavily.
  - Optimization uses Gradient Descent or similar algorithms.
- Key advantage :
  - Correct predictions → low loss.
  - Wrong predictions → high loss, especially if predicted probability was high.

Q. : How entropy comes in this picture ? Why the name "Cross-Entropy" ?

What about classification in multiple classes/categories ?

Multi class classification

- Binary logistic regression → two classes (0 or 1).
- For multiple classes (K classes), two main approaches:
  - One-vs-Rest (OvR):
    - Train K separate binary classifiers.
    - Pick the class with highest probability.
  - One-vs-One (OvO):
    - Train a classifier for each pair of classes.
    - Use voting to decide final class.



Multi class classification with SoftMax

- SoftMax Regression:
  - Generalization of logistic regression for multi-class.
  - Computes probability for each class:

$$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_{j=1}^{K} e^{w_j^T x}}$$

$$\boxed{z_k = w_k^T x}$$

  - Predictions = Class with highest probability.
  - Note that, Logistic regression scales well to multiple classes but stays linear in decision boundaries.

- SoftMax ensures the following :
  - All probabilities are between 0 and 1.
  - Sum of probabilities = 1.
  - Larger z_k → higher probability for class k.

- Prediction = Class with maximum probability.

Multi class classification with SoftMax

- ***SoftMax is like applying sigmoid but extended to multiple classes.***
- Turns raw scores into something we can interpret as probabilities.

- Example:

  Three classes with raw scores [2, 1, 0].

  Apply SoftMax → Probabilities [0.71, 0.26, 0.03].

  Note that probabilities are adding up to 1, which is important.

Now, the main question :
How do we know how well my model is performing ?

Or

How to evaluate my model in classification ?

The Confusion Matrix

A key diagnostic tool for evaluating classification tasks

Several evaluation metric can be calculated based on confusion matrix

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	**Positive**	True Positive (TP)	False Negative (FN) Type II Error	**Sensitivity** $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

The F1 - score

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

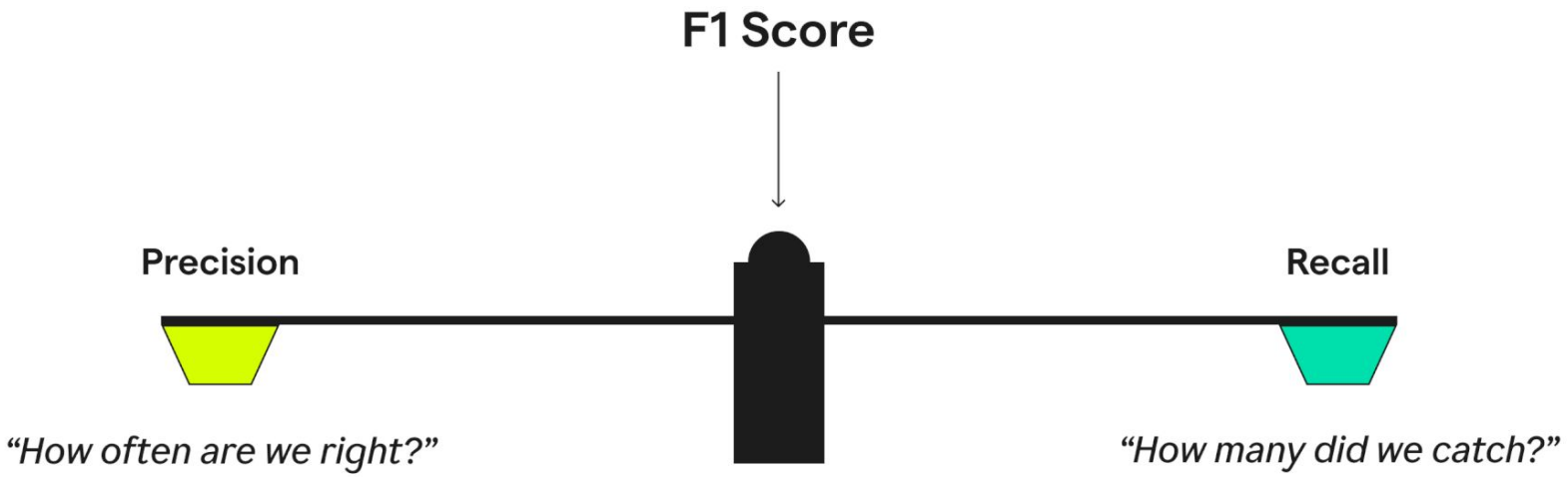


Image credit : grammarly.com

Overall

- **Accuracy**: Overall correctness

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision**: How many predicted positives are correct

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity)**: How many actual positives are found

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score**: Balance between Precision & Recall

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Points to be noted

- Accuracy can be misleading with imbalanced data.
- Precision vs Recall = trade-off (depends on application).
- F1-score useful when classes are imbalanced.

Q. : What do we mean by imbalanced data ?

Example

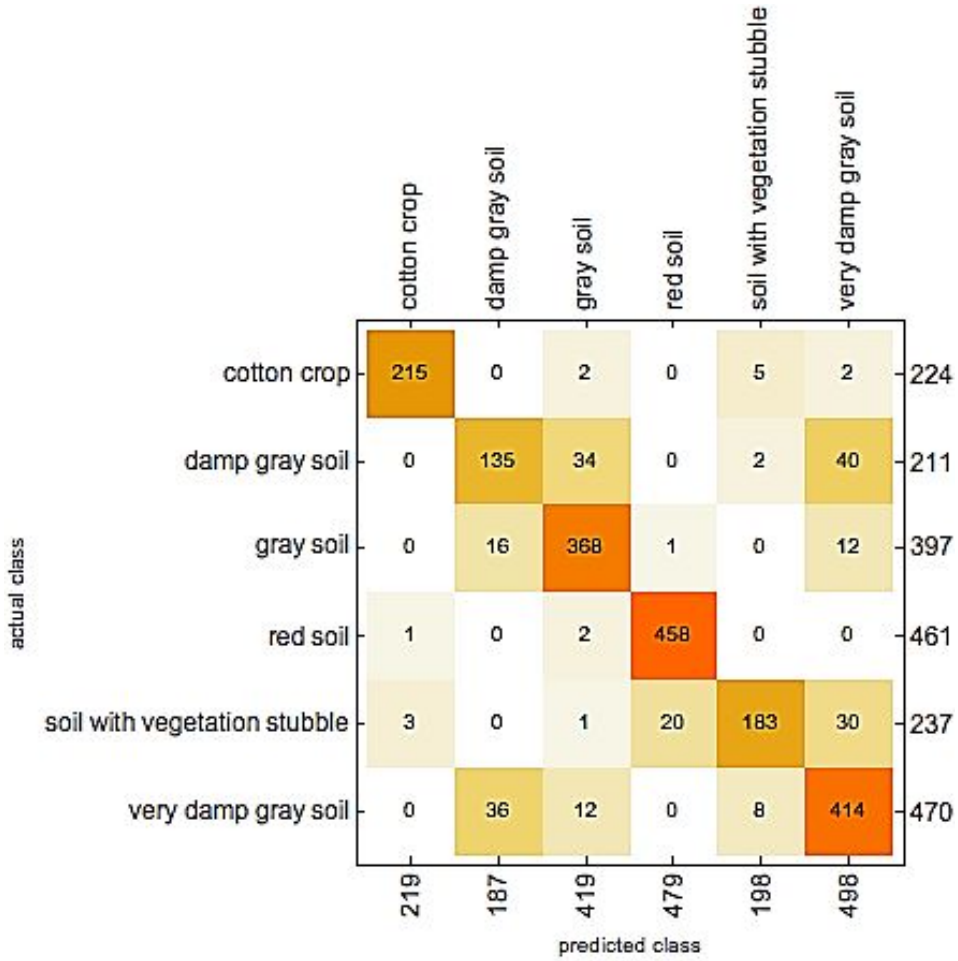
		Predicted	
		Spam	Non-spam
Actual	Spam	600	300
	Non-spam	100	9000

A ML model was used to classify 10,000 emails in “spam” or “Non-spam”. The confusion matrix for the same is given here. Evaluate the model performance by calculating :

a) Precision
b) Recall
c) Accuracy
d) F1-score

Confusion matrix for multi-class classification

- The logic and the principles are same as that of binary classification.
- Each class treated as “positive” in a one-vs-all manner.
- Metrics (Precision, Recall, F1 etc.) need averaging strategies:
  - **Macro Average:** Equal weight to each class.
  - **Micro Average:** Weighted by sample count.



Averaging strategies

Macro Averaging (treat all classes equally)

- Compute Precision/Recall/F1 for each class.
- Then take the average across all classes.
- Gives **equal importance to each class**.
- Good when classes are balanced.
- Formula :

$$Macro\ Precision = \frac{1}{K}\sum_{k=1}^{K} Precision_k$$

Micro Averaging (treat all samples equally)

- Aggregate TP, FP, FN across all classes.
- Then compute Precision/Recall/F1 from totals.
- Gives **more weight to larger classes**.
- Good when classes are imbalanced.
- Formula :

$$Micro\ Precision = \frac{\sum_{k=1}^{K} TP_k}{\sum_{k=1}^{K}(TP_k + FP_k)}$$

Examples

Consider the following data for a classification tasks where classes are : Cat, Dog, Rabbit

Classes	TP	FP	FN
Cat	50	10	5
Dog	40	5	15
Rabbit	10	20	10

Step 1: Per-class Precision

- Cat: $\frac{50}{50+10} = 0.83$
- Dog: $\frac{40}{40+5} = 0.89$
- Rabbit: $\frac{10}{10+20} = 0.33$

Macro Precision = (0.83 + 0.89 + 0.33) / 3 = 0.68

Step 2: Micro Precision

- Total TP = 50 + 40 + 10 = 100
- Total FP = 10 + 5 + 20 = 35
- Micro Precision = $\frac{100}{100+35} = 0.74$

- Macro treats Rabbit equally, so overall score drops (0.68).
- Micro weights by total samples, so strong Cat/Dog performance dominates (0.74).

Overview

- From Regression to Classification
  - Linear regression unsuitable for classification → predictions unbounded.
  - Logistic regression adds sigmoid to output probabilities (0–1).
  - Uses odds & logit for linear modelling of probabilities.
- Model & Training
  - Decision boundary at $p = 0.5$.
  - Parameters estimated via Maximum Likelihood Estimation (MLE).
  - Cost Function: Cross-Entropy (Log Loss).
- Extensions
  - Multi-class classification: One-vs-Rest, One-vs-One, SoftMax.
  - SoftMax generalizes logistic regression to multiple classes.
- Evaluation
  - Confusion Matrix: Foundation for metrics.
  - Metrics: Accuracy, Precision, Recall, F1.
  - Multi-class evaluation needs Macro vs Micro averaging.